OPLOGS-99: Best Practice Summary

> **Series:** OPLOGS — OpenPipeline Logs | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/23/2026

Definitive best practice settings for OpenPipeline log processing. Each entry specifies the exact configuration — no hedging, no options.

Table of Contents

1. Pipeline Configuration
2. Bucket Design & Retention
3. Data Masking
4. DQL Query Patterns
5. Metric Extraction
6. Cost Optimization
7. Security & Access Control
8. Monitoring & Alerting
9. DQL Cookbook

---

1. Pipeline Configuration

Practice	Recommended Setting/Value	Priority
Process data at ingestion, not query time	Configure parsing, enrichment, masking, and routing as OpenPipeline processors	Critical

Practice	Recommended Setting/Value	Priority
Processor order within the Processing stage	Place masking processors before any processor that copies or parses the sensitive field (e.g., mask `content` before a DQL `parse` that extracts from it); order within a stage is the order you configure — each processor's output is the next one's input	Critical
First-match order	Routes (which pick the **pipeline**) and Bucket assignment processors (which pick the **bucket**) are both first-match: most specific first, catch-all last	Critical
Drop health check logs at ingestion	Drop record processor: `matchesValue(content, "*health*") AND loglevel == "INFO"` — `contains()` is not enabled in OpenPipeline matchers	Recommended
Drop DEBUG logs in production	Drop processor: `loglevel == "DEBUG"` — or, to keep extraction from DEBUG records, a **No storage assignment** processor in the Bucket assignment stage (there is no log-sampling processor)	Recommended
Parse NONE-level logs	DQL processor with matcher `loglevel == "NONE"`: `fieldsAdd parsed_levels = parseAll(content, "'[' LD:l ']'")`, then set `loglevel` to the first element whose `upper()` is a supported level (EMERGENCY, ALERT, CRITICAL, SEVERE, ERROR, FATAL, WARN, NOTICE, INFO, DEBUG, TRACE) — a leading `[timestamp][thread]` does not hide the level, and anything else leaves the record unchanged. Not `DATA? '[' LD ']'`, which captures a later bracket	Recommended
Add computed environment attribute	`fieldsAdd environment = if(contains(k8s.namespace.name, "prod"), "production", else: "development")`	Recommended
Confirm nothing is left on the classic pipeline	`countIf(in(dt.openpipeline.pipelines, "logs:default"))` must be 0 — `isNotNull(dt.openpipeline.pipelines)` is true for classic records too, so it proves nothing	Critical

Practice	Recommended Setting/Value	Priority
Compare hourly volume before/after migration	`makeTimeseries count(), by: {dt.openpipeline.source}, interval:1h` — volume must be consistent	Critical

2. Bucket Design & Retention

Practice	Recommended Setting/Value	Priority
Create purpose-driven buckets	Minimum: `default_logs`, `debug_logs`, `error_logs`, `audit_logs`, `security_logs`	Critical
Bucket naming convention	`<purpose>_logs` or `<environment>_<purpose>_logs`	Recommended
DEBUG/TRACE log retention	7 days	Critical
Standard INFO log retention	35 days	Recommended
ERROR log retention	90 days	Recommended
Audit log retention	365 days (730 for regulated industries)	Critical
Security log retention	180 days	Recommended
Verify retention compliance	`fetch dt.system.buckets | filter dt.system.table == "logs" | fields name, retention_days` — audit buckets must show `retention_days >= 365`	Critical

3. Data Masking

Practice	Recommended Setting/Value	Priority
Mask in the Processing stage, before anything copies the field	OpenPipeline masking is a DQL processor in the first stage (Processing); records reach Bucket assignment and storage only after it — but a processor placed *before* the masking processor sees the raw value	Critical
Mask email addresses	Pattern: `[A-Za-z0-9._%+-]+ '@' [A-Za-z0-9.-]+` in a DQL processor (`replacePattern`) → `[EMAIL-MASKED]`	Critical
Mask credit card numbers	Pattern: `!<<[0-9A-Za-z-] CREDITCARD !>>[0-9A-Za-z-]` (bounded — unbounded `CREDITCARD` rewrites digit runs inside UUIDs) in a DQL processor (`replacePattern`) → `[CC-MASKED]`	Critical
Mask SSNs	Pattern: `[0-9]{3} '-' [0-9]{2} '-' [0-9]{4}` in a DQL processor (`replacePattern`) → `[SSN-MASKED]`	Critical
Mask API keys and tokens	Pattern: `<<(('api_key' | 'API_KEY' | 'apikey' | 'APIKEY' | 'key' | 'KEY' | 'token' | 'TOKEN') '=') [A-Za-z0-9_-]{20,}` , plus a second statement for JSON-quoted keys — DPL literals are case-sensitive, so list each casing; include `_` and `-` , or keys like `sk_live_…` stay in clear in a DQL processor (`replacePattern`) → `[KEY-MASKED]`	Critical
Mask JWT tokens	Search for `eyJ` prefix, mask full token value	Critical
Mask passwords	Pattern: `password=` , `passwd=` , `pwd=` in content	Critical
Hash instead of mask when correlation is needed	DQL processor: `fieldsAdd user_hash = hashSha256(user.email) | fieldsRemove user.email`	Recommended

Practice	Recommended Setting/Value	Priority
Run sensitive data discovery weekly	Search for `@`, `key=`, `token=`, `password`, `eyJ`, `card` patterns	Recommended
Run a leak check after every masking change	`countIf(replacePattern(content, "<masking pattern>", "") != content)` must be 0 — it finds a value the masking pattern would still catch, anywhere in the line	Critical
Run monthly PII exposure audit	Track `email_exposure_pct`, `key_exposure_pct`, `password_exposure_pct` — all must trend toward 0%	Critical

Masking patterns are DPL, not regex (FAQ-15 §3) — regex syntax such as `\b`, `\d` or `$1` is rejected when the processor is saved.

> **Sources:** OpenPipeline processing examples (DT docs) — *"You can mask parts of an attribute by leveraging replacePattern in combination with other DQL functions."*, Processing in OpenPipeline (DT docs) — *"each processor output becomes the input for the next one"*

4. DQL Query Patterns

Practice	Recommended Setting/Value	Priority
Always specify time range	`fetch logs, from:-1h` — never bare `fetch logs`	Critical
Filter early, sort/limit last	`filter` immediately after `fetch`; `sort` and `limit` as final commands	Critical
Target specific buckets	`fetch logs, bucket:{"error_logs"}, from:-7d` — an unscoped query reads every log bucket. Use the `{...}` set form: a bare string runs but raises an INFO notice asking for curly braces	Critical

Practice	Recommended Setting/Value	Priority
Use `==` for exact matches	`filter field == "value"` — reserve `~` for wildcards only	Recommended
Always alias aggregations	`summarize count = count()` then `sort count desc` — anonymous aggregations cannot be referenced	Critical
Use `isNull()` / `isNotNull()`	Never `== null` — DQL uses three-valued logic	Critical
Use `in()` with curly-brace arrays	`in(status, {"ERROR", "WARN"})` — values are uppercase; never SQL-style `IN ('a', 'b')`	Critical
Use `coalesce()` for fallbacks	`coalesce(k8s.workload.name, k8s.namespace.name, "UNKNOWN")` — falls back where a field is null; `loglevel` is never null (missing levels are `"NONE"`), so it is a poor first argument	Recommended
Use `entityName()` for display names	`entityName(dt.entity.host, type:"dt.entity.host")` — a readable name instead of an ID; note `dt.entity.*` fields are `deprecated` in favour of `dt.smartscape.*` (OPLOGS-06)	Recommended
Filter `isNotNull` before aggregating optional fields	Prevents null group-by values in aggregations	Recommended
Use `samplingRatio` for exploration	`fetch logs, from:-1d, samplingRatio:10` then multiply back	Recommended
Bound one query's scan with `scanLimitGBytes`	`fetch logs, scanLimitGBytes:100` — whole GB only. The query stops at the limit and may return partial results (`SCAN_LIMIT_GBYTES` warning); it is a safety stop, not a budget. Avoid `-1` (no limit)	Optional

5. Metric Extraction

Practice	Recommended Setting/Value	Priority
Extract dimensional metrics at ingestion	Create `log.request.count`, `log.response.duration`, `log.error.count` with service/method/status dimensions	Recommended
Minimum metric dimensions	`k8s.namespace.name`, `k8s.workload.name`, `loglevel`	Recommended
Use reverse-DNS for business event types	`com.example.payment.completed` format	Optional
Generate business events from log patterns	Order completion, user signup, deployment events	Optional

6. Cost Optimization

Practice	Recommended Setting/Value	Priority
Calculate droppable log percentage weekly	Health check + heartbeat + debug as % of total — the share you can drop is specific to your estate; measure before setting a target	Recommended
Measure storage by log level daily	`fieldsAdd content_bytes = stringLength(content)` then `summarize sum(content_bytes)/1048576.0, by:{loglevel}`	Recommended
Track bucket volume distribution weekly	Query all buckets showing total logs, unique sources, debug %, error % per bucket	Recommended

7. Security & Access Control

Practice	Recommended Setting/Value	Priority
Bucket-level IAM policies	`ALLOW storage:buckets:read WHERE storage:bucket-name="audit_logs"; ALLOW storage:logs:read WHERE storage:bucket-name="audit_logs";`	Critical
Default bucket: all-authenticated read	Grant `storage:logs:read` on `default_logs` to all authenticated users	Recommended
Audit bucket: restricted access	`storage:logs:read` on `audit_logs` exclusively to audit team group	Critical
Classify IPs as internal vs external	Extract with `parseAll(content, "IPADDR:ip")`, classify with `ipIn(ip, {"10.0.0.0/8", "172.16.0.0/12", "192.168.0.0/16"})`	Recommended
Monitor failed auth attempts	`(contains(content, "failed") OR contains(content, "denied")) AND (contains(content, "login") OR contains(content, "auth"))` at 30-min intervals	Critical

8. Monitoring & Alerting

Practice	Recommended Setting/Value	Priority
Error rate alert threshold	>5% per entity over 15-minute window, minimum 100 records	Recommended
Error count threshold	>10 per namespace per 15 minutes	Recommended
Pod crash loop detection	`dt.kubernetes.container.restarts` summed per pod, alert when >3 in 1 hour — log lines saying "started" do not count restarts	Recommended

Practice	Recommended Setting/Value	Priority
Security error trends	`makeTimeseries count(), interval:30m` filtered on unauthorized/forbidden/denied	Recommended
Weekly bucket health check	Total logs, unique sources, debug %, error % per bucket over 7 days	Recommended

---

9. DQL Cookbook

Reference queries consolidated from across the OPLOGS series — copy / paste into your tenant for dashboard tiles, monitoring widgets, or SLO calculations. These were previously embedded in OPLOGS-07 (Analytics & Dashboards); they live here so the main analytics notebook stays focused on the *how* of building queries rather than serving as a query repository.

9.1 Dashboard-Ready Queries

Single-value, time-series, breakdown, and percentile queries optimized for dashboard tiles. Drop them directly into a Dashboard tile or Notebook visualization.

```
// Single Value: Total Logs (Last Hour)
fetch logs, from: now() - 1h
| summarize {total_logs = count()}
```

```
// Single Value: Error Count (Last Hour)
// status == "ERROR" covers every error-class level (ERROR, SEVERE, CRITICAL,
FATAL, …);
// loglevel == "ERROR" alone misses SEVERE and the rest
fetch logs, from: now() - 1h
| filter status == "ERROR"
| summarize {error_count = count()}
```

```
// Pie Chart: Log Level Distribution
fetch logs, from: now() - 1h
| summarize {count = count()}, by: {loglevel}
| sort count desc
```

```
// Bar Chart: Top Error Sources
fetch logs, from: now() - 1h
| filter status == "ERROR"
| summarize {error_count = count()}, by: {k8s.namespace.name}
| sort error_count desc
| limit 10
```

```
// Line Chart: Log Trend (6 Hours)
fetch logs, from: now() - 6h
| makeTimeseries {
    errors = countIf(status == "ERROR"),
    warnings = countIf(status == "WARN"),
    info = countIf(status == "INFO")
  }, interval: 5m
```

```
// Table: Top Error Messages
fetch logs, from: now() - 1h
| filter status == "ERROR"
| fieldsAdd error_preview = substring(content, from: 0, to: 100)
| summarize {
    occurrences = count(),
    first_seen = min(timestamp),
    last_seen = max(timestamp)
  }, by: {error_preview, k8s.namespace.name}
| sort occurrences desc
| limit 20
```

```
// Heat Map: Errors by Hour and Namespace
fetch logs, from: now() - 24h
| filter status == "ERROR"
| fieldsAdd hour_bucket = bin(timestamp, 1h)
| summarize {error_count = count()}, by: {hour_bucket, k8s.namespace.name}
| sort hour_bucket asc
```

9.2 Operational Dashboard Queries

SLO and operational queries: error budgets, top-N services by log volume, per-host log distribution. Designed for an ops-on-call dashboard.

```
// Operations Summary (last 1h)
fetch logs, from: now() - 1h
| summarize {
    total_logs = count(),
    error_count = countIf(status == "ERROR"),
    warn_count = countIf(status == "WARN"),
    unique_hosts = countDistinct(dt.entity.host),
    unique_pods = countDistinct(k8s.pod.name)
  }
| fieldsAdd error_rate = round((error_count * 100.0) / total_logs, decimals:
2)
```

```
// Health scorecard by namespace
fetch logs, from: now() - 1h
| filter isNotNull(k8s.namespace.name)
| summarize {
    total = count(),
    errors = countIf(status == "ERROR"),
    warnings = countIf(status == "WARN")
  }, by: {k8s.namespace.name}
| fieldsAdd error_rate = round((errors * 100.0) / total, decimals: 2)
| fieldsAdd health_status = if(error_rate &gt; 10, "CRITICAL",
                            else: if(error_rate &gt; 5, "WARNING",
                            else: "HEALTHY"))
| sort error_rate desc
```

```
// Ingestion pipeline summary
fetch logs, from: now() - 1h
| summarize {
    log_count = count(),
    unique_pipelines = countDistinct(dt.openpipeline.pipelines)
  }, by: {dt.openpipeline.source, dt.system.bucket}
| sort log_count desc
```

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.